Homework 6 Report
Wilson Neira

Part 1: Reading!

note @687

Thoughts on Parnas Paper

Updated 3 days ago by Wilson Neira

What I liked about this reading is that it shows people were already thinking seriously about "how to split a system" back in 1972, and Parnas doesn't just say use modules, he explains that the criteria you use to divide modules is what really determines whether the system becomes flexible, understandable, and easier to develop.

What I recognize as related to microservice architectures is his "information hiding" idea is that instead of splitting by workflow steps (like a flowchart), you split by design decisions that might change and hide them behind simple interfaces, which is basically what we try to do with services today so one service can change internally without forcing every other service to change too.

parnas_paper

Thoughts on Parnas Paper

Updated 3 days ago by Mayank Tamakuwala

One thing I really liked about this reading was how it challenges the traditional way of breaking systems into modules. Instead of organizing modules based on steps in a flowchart (like input, processing, and output), Parnas argues that modules should be built around design decisions that are likely to change and that each module should hide those decisions from the rest of the system. The comparison between the two KWIC decompositions made this very clear: in the second version, details like storage format or circular shift logic were hidden inside specific modules, which made the system much easier to modify without affecting everything else. I thought this was powerful because it shows that good modularization is really about managing change, not just organizing code.

This connects directly to microservice architecture. In microservices, each service encapsulates its own data and internal logic, and other services interact with it only through well-defined APIs. That's basically information hiding at a distributed systems level. Just like in the second decomposition, internal changes within a service (like database structure or processing logic) shouldn't impact other services as long as the interface remains stable. The reading also highlights how abstract interfaces support independent development, which mirrors how different teams can own and deploy microservices independently.

parnas_paper

Edit 1 17 views

1 Followup Discussions

Resolved @685_f1

Wilson Neira 3 days ago

Hi Mayank Tamakuwala, I agree because modular design is really about hiding design decisions so the system can change more easily, which is the same mindset we use in microservices where each service hides its internal logic behind a stable API.

1

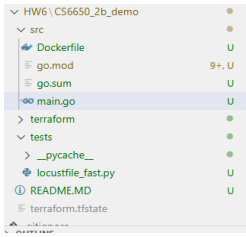
Part 2: Identifying Performance Bottlenecks!

1. The Question: But in the real world, when your service slows down, how do you know if you need better code or just more servers?

When my service slows down, I can check if CPU/memory is maxed out during load tests (meaning I need more servers/resources) or if the code is doing unnecessary work even when resources aren't close to full (meaning I need to optimize the code).

2. Starting Infrastructure (CS6650_2b_demo already has required ECS Fargate Configuration)

3. Implementation Requirements (Product Structure, Data Generation, Search Logic, Response Format)




```go
type Product struct {
    ID          int    `json:"id"`
    Name        string `json:"name"`
    Category    string `json:"category"`
    Description string `json:"description"`
    Brand       string `json:"brand"`
}
```

```
Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems (main)
$ curl "http://localhost:8080/products/search?q=electronics"
curl "http://localhost:8080/health"
{"products":[{"id":87032,"name":"Product Alpha 87032","category":"Electronics","description":"Budget-friendly pick","brand":"Alpha"},{"id":20736,"name"
:"Product Alpha 20736","category":"Electronics","description":"High performance option","brand":"Alpha"},{"id":46504,"name":"Product Alpha 46504","cate
gory":"Electronics","description":"Popular choice for most users","brand":"Alpha"},{"id":92800,"name":"Product Alpha 92800","category":"Electronics","d
escription":"Everyday quality item","brand":"Alpha"},{"id":47776,"name":"Product Alpha 47776","category":"Electronics","description":"High performance
option","brand":"Alpha"},{"id":59624,"name":"Product Alpha 59624","category":"Electronics","description":"Popular choice for most users","brand":"Alpha
"},{"id":62624,"name":"Product Alpha 62624","category":"Electronics","description":"Popular choice for most users","brand":"Alpha"},{"id":50160,"name":
"Product Alpha 50160","category":"Electronics","description":"Everyday quality item","brand":"Alpha"},{"id":41672,"name":"Product Alpha 41672","categor
y":"Electronics","description":"Budget-friendly pick","brand":"Alpha"},{"id":43232,"name":"Product Alpha 43232","category":"Electronics","description":
"Budget-friendly pick","brand":"Alpha"},{"id":99632,"name":"Product Alpha 99632","category":"Electronics","description":"Budget-friendly pick","brand":
"Alpha"},{"id":54264,"name":"Product Alpha 54264","category":"Electronics","description":"Popular choice for most users","brand":"Alpha"},{"id":53792,"
name":"Product Alpha 53792","category":"Electronics","description":"Budget-friendly pick","brand":"Alpha"},{"id":55184,"name":"Product Alpha 55184","ca
tegory":"Electronics","description":"Popular choice for most users","brand":"Alpha"},{"id":44352,"name":"Product Alpha 44352","category":"Electronics",
"description":"Budget-friendly pick","brand":"Alpha"},{"id":87608,"name":"Product Alpha 87608","category":"Electronics","description":"Premium build an
d feel","brand":"Alpha"},{"id":22296,"name":"Product Alpha 22296","category":"Electronics","description":"High performance option","brand":"Alpha"},{"i
d":15760,"name":"Product Alpha 15760","category":"Electronics","description":"Everyday quality item","brand":"Alpha"}],"total_found":18,"search_time":"
0s"}
ok
```

4. Deploy Terraform

terraform init

terraform apply -auto-approve

```
Apply complete! Resources: 6 added, 0 changed, 1 destroyed.

Outputs:

ecs_cluster_name = "CS6650L2-cluster"
ecs_service_name = "CS6650L2"
```

5. Get public IP and Test

3.229.117.16

```
$ curl "http://3.229.117.16:8080/products/search?q=electronics"
curl "http://3.229.117.16:8080/health"
{"products":[{"id":2976,"name":"Product Alpha 2976","category":"Electronics","description":"High performance option","brand":"Alpha"},{"id":84120,"name
":"Product Alpha 84120","category":"Electronics","description":"Everyday quality item","brand":"Alpha"},{"id":15032,"name":"Product Alpha 15032","categ
ory":"Electronics","description":"Budget-friendly pick","brand":"Alpha"},{"id":35840,"name":"Product Alpha 35840","category":"Electronics","description
":"Everyday quality item","brand":"Alpha"},{"id":23152,"name":"Product Alpha 23152","category":"Electronics","description":"Budget-friendly pick","bran
d":"Alpha"},{"id":42392,"name":"Product Alpha 42392","category":"Electronics","description":"Budget-friendly pick","brand":"Alpha"},{"id":4704,"name":"
Product Alpha 4704","category":"Electronics","description":"Popular choice for most users","brand":"Alpha"},{"id":12152,"name":"Product Alpha 12152","c
ategory":"Electronics","description":"Budget-friendly pick","brand":"Alpha"},{"id":39968,"name":"Product Alpha 39968","category":"Electronics","descrip
tion":"Premium build and feel","brand":"Alpha"},{"id":8216,"name":"Product Alpha 8216","category":"Electronics","description":"High performance option"
,"brand":"Alpha"}],"total_found":10,"search_time":"307.074µs"}
ok
```

6. Load Testing & Analysis (FastHttpUser) User Product Search with Search Terms

locustfile_fast.py

7. Implementation Verification: 1 user 1 minute

locust -f locustfile_fast.py --host http://3.229.117.16:8080

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	3028	0	19	26	33	19.81	10	336	1641.06	49.9	0
	Aggregated	3028	0	19	26	33	19.81	10	336	1641.06	49.9	0

8. Test 1 - Baseline: 5 users for 2 minutes

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	28361	0	20	29	39	20.57	10	180	1624.1	245	0
	Aggregated	28361	0	20	29	39	20.57	10	180	1624.1	245	0

9. Test 2 - Breaking Point: 20 users for 3 minutes

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	161457	0	19	26	89	20.41	10	236	1624.33	944.9	0
	Aggregated	161457	0	19	26	89	20.41	10	236	1624.33	944.9	0

10. Which resource hits the limit first?

CPU hits the limit first because its usage spikes very high while memory stays low.

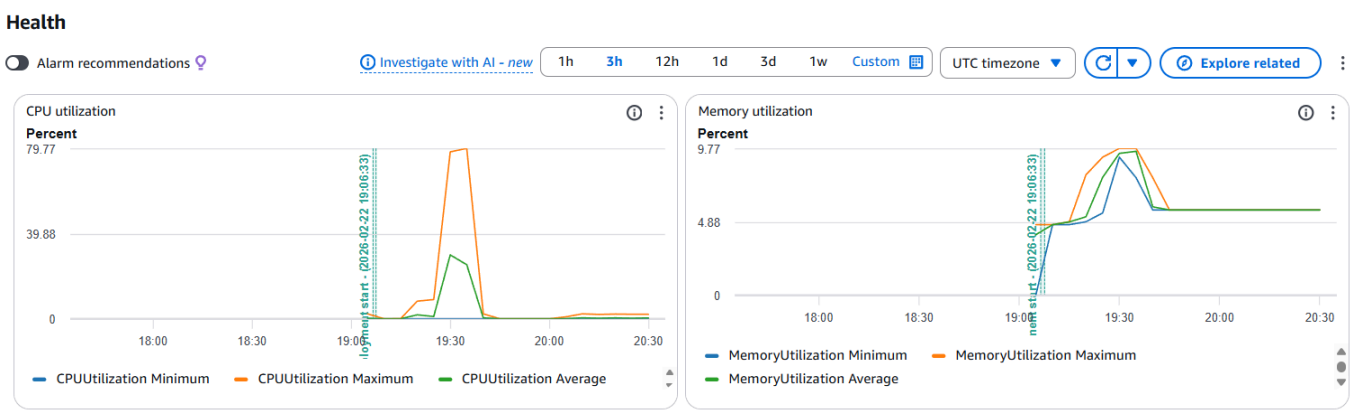
11. How much did response times degrade?

Response times only got a little slower, increasing slightly but still staying fairly fast overall.

12. Could you solve this by doubling CPU (256 → 512 units)?

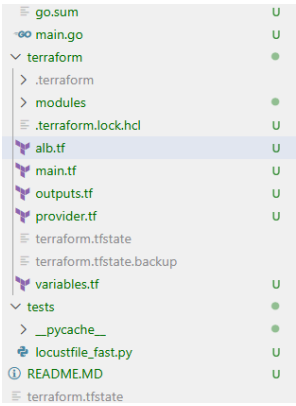
Yes, since CPU is the bottleneck, giving the service more CPU would likely handle the load better.

13. CloudWatch Monitoring



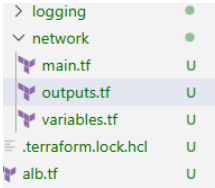
Part III: Horizontal Scaling with Auto Scaling

1. Application Load Balancer (ALB)



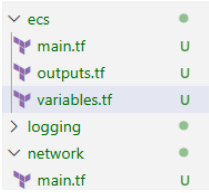

```tf
# --- Application Load Balancer ---
resource "aws_lb" "app" {
  name               = "${var.service_name}-alb"
  load_balancer_type = "application"
  security_groups    = [aws_security_group.alb_sg.id]
  subnets            = module.network.subnet_ids
}

# --- Target Group (Fargate requires target_type = ip)
resource "aws_lb_target_group" "app_tg" {
```

2. module.network outputs vpc_id



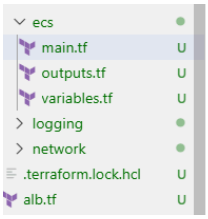

```tf
output "vpc_id" {
  description = "VPC id"
  value       = data.aws_vpc.default.id
}
```

3. Update ECS module to attach the Target Group




```tf
variable "target_group_arn" {
  type        = string
  description = "ALB target group ARN"
}
```

4. Add load_balancer {} inside aws_ecs_service.this

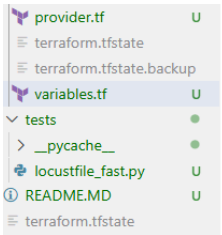



```tf
  load_balancer {
    target_group_arn = var.target_group_arn
    container_name   = "${var.service_name}-container"
    container_port   = var.container_port
  }
```

5. New variable into terraform/main.tf inside module "ecs"

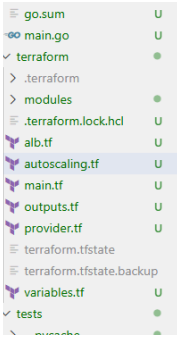
```tf
  region             = var.aws_region
  target_group_arn = aws_lb_target_group.app_tg.arn
}
```

6. In variables.tf set ecs_count default to 2




```tf
variable "ecs_count" {
  type    = number
  default = 2
  description = "Desired Fargate task count"
}
```

7. Auto Scaling




```
target_tracking_scaling_policy_configuration {
  predefined_metric_specification {
    predefined_metric_type = "ECSServiceAverageCPUUtilization"
  }

  target_value       = 70
  scale_in_cooldown  = 300
  scale_out_cooldown = 300
}
```

8. Deploy Terraform and Test with alb_dns_name

terraform apply -auto-approve

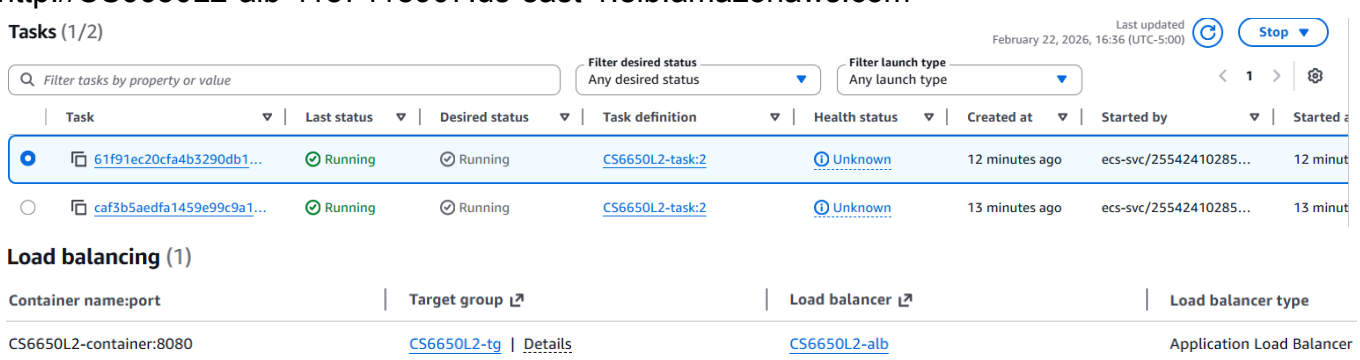
terraform output -raw alb_dns_name

alb_dns_name = "CS6650L2-alb-1187418907.us-east-1.elb.amazonaws.com"

```
$ curl "http://CS6650L2-alb-1187418907.us-east-1.elb.amazonaws.com/health"
curl "http://CS6650L2-alb-1187418907.us-east-1.elb.amazonaws.com/products/search?q=electronics"
ok{"products":[{"id":34384,"name":"Product Alpha 34384","category":"Electronics","description":"Popular choice for most users","brand":"Alpha"},{"id":94808,"name":"
Product Alpha 94808","category":"Electronics","description":"Premium build and feel","brand":"Alpha"},{"id":3776,"name":"Product Alpha 3776","category":"Electronics
","description":"High performance option","brand":"Alpha"},{"id":51928,"name":"Product Alpha 51928","category":"Electronics","description":"Premium build and feel",
"brand":"Alpha"},{"id":96888,"name":"Product Alpha 96888","category":"Electronics","description":"Premium build and feel","brand":"Alpha"},{"id":24664,"name":"Produ
ct Alpha 24664","category":"Electronics","description":"Popular choice for most users","brand":"Alpha"},{"id":82824,"name":"Product Alpha 82824","category":"Electro
nics","description":"Popular choice for most users","brand":"Alpha"},{"id":70880,"name":"Product Alpha 70880","category":"Electronics","description":"Everyday quali
ty item","brand":"Alpha"},{"id":56568,"name":"Product Alpha 56568","category":"Electronics","description":"Premium build and feel","brand":"Alpha"},{"id":84888,"nam
e":"Product Alpha 84888","category":"Electronics","description":"Premium build and feel","brand":"Alpha"},{"id":14896,"name":"Product Alpha 14896","category":"Elect
ronics","description":"High performance option","brand":"Alpha"},{"id":6496,"name":"Product Alpha 6496","category":"Electronics","description":"High performance opt
ion","brand":"Alpha"},{"id":18208,"name":"Product Alpha 18208","category":"Electronics","description":"Premium build and feel","brand":"Alpha"},{"id":32800,"name":"
Product Alpha 32800","category":"Electronics","description":"Everyday quality item","brand":"Alpha"}],"total_found":14,"search_time":"118.636µs"}
```

9. The Core Test

locust -f locustfile_fast.py --host http://CS6650L2-alb-1187418907.us-east-1.elb.amazonaws.com



10. Test 1 - Baseline: 5 users for 2 minutes

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	25643	0	21	34	47	22.89	10	339	1624.96	219.7	0
	Aggregated	25643	0	21	34	47	22.89	10	339	1624.96	219.7	0

Tasks (1/2)

Last updated February 22, 2026, 16:44 (UTC-5:00) | Stop

Filter tasks by property or value | Filter desired status: Any desired status | Filter launch type: Any launch type

Task	Last status	Desired status	Task definition	Health status	Created at	Started by	Started a
61f91ec20cfa4b3290db1...	Running	Running	CS6650L2-task:2	Unknown	20 minutes ago	ecs-svc/25542410285...	20 minut
caf3b5aedfa1459e99c9a1...	Running	Running	CS6650L2-task:2	Unknown	21 minutes ago	ecs-svc/25542410285...	20 minut

Details

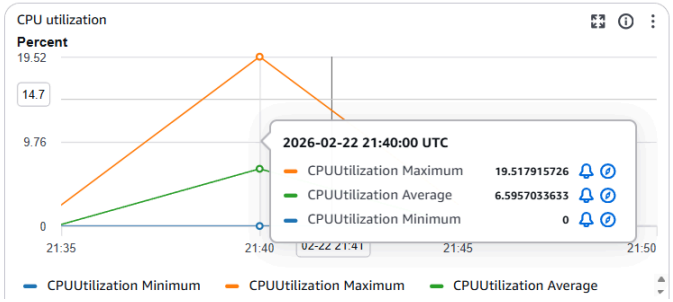
arn:aws:elasticloadbalancing:us-east-1:316009999564:targetgroup/CS6650L2-tg/050770c8ab473a84

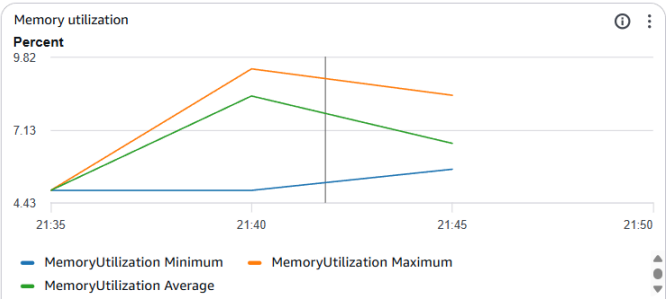
Target type	Protocol : Port	Protocol version	VPC
IP	HTTP: 8080	HTTP1	vpc-0cea8d47d1bab81b6
IP address type	**Load balancer**		
IPv4	CS6650L2-alb		

Total targets	Healthy	Unhealthy	Unused	Initial	Draining
2	2	0	0	0	0
	0 Anomalous				

Health

Alarm recommendations | Investigate with AI - *new* | 1h 3h 12h 1d 3d 1w Custom (15m) | UTC timezone | Explore related





11. Test 2 - Breaking Point: 20 users for 3 minutes

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	149963	0	20	32	61	22.07	10	226	1626.35	938.5	0
	Aggregated	149963	0	20	32	61	22.07	10	226	1626.35	938.5	0

Tasks (1/2)

Last updated February 22, 2026, 17:00 (UTC-5:00) | Stop

Filter tasks by property or value | Filter desired status: Any desired status | Filter launch type: Any launch type

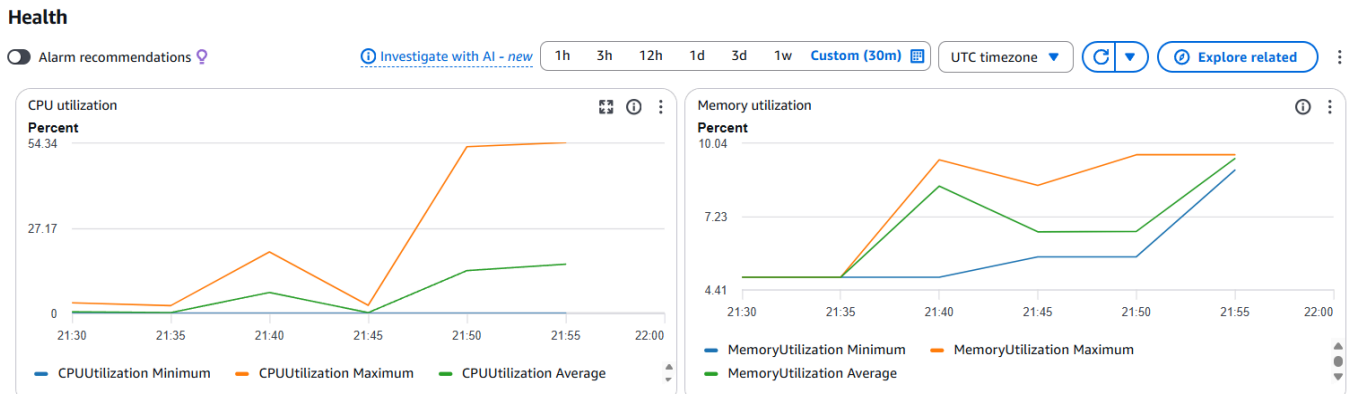
Task	Last status	Desired status	Task definition	Health status	Created at	Started by	Started a
61f91ec20cfa4b3290db1...	Running	Running	CS6650L2-task:2	Unknown	36 minutes ago	ecs-svc/25542410285...	36 minut
caf3b5aedfa1459e99c9a1...	Running	Running	CS6650L2-task:2	Unknown	36 minutes ago	ecs-svc/25542410285...	36 minut

Details

arn:aws:elasticloadbalancing:us-east-1:316009999564:targetgroup/CS6650L2-tg/050770c8ab473a84

Target type	Protocol : Port	Protocol version	VPC
IP	HTTP: 8080	HTTP1	vpc-0cea8d47d1bab81b6
IP address type	**Load balancer**		
IPv4	CS6650L2-alb		

Total targets	Healthy	Unhealthy	Unused	Initial	Draining
2	2	0	0	0	0
	0 Anomalous				



12. Response times compared to Part A?

Response times stayed about the same or slightly higher than Part A, but still fast and stable even under heavy load.

13. System availability during scaling events?

The system stayed available the whole time and did not go down while new instances were starting.

14. How does the system respond to the load that broke Part A?

Instead of slowing down badly, the system handled the load by spreading requests across multiple instances.

15. When do new instances get added?

New instances get added when CPU usage gets high and stays above the scaling threshold.

16. How is the load distributed across instances?

The load balancer sends requests evenly to all healthy instances so no single one gets overloaded.

17. What happens to response times as instances scale?

Response times improve or stay steady as more instances come online because the work is shared.

18. Resilience Testing Test 2 - Breaking Point: 20 users for 3 minutes

Task	Last status	Desired status	Task definition	Health status	Created at	Started by	Started a
caf3b5aedfa1459e99c9a1...	Running	Running	CS6650L2-task:2	Unknown	44 minutes ago	ecs-svc/25542410285...	44 minut
61f91ec20cfa4b3290db1...	Deactivating	Stopped	CS6650L2-task:2	Unknown	44 minutes ago	ecs-svc/25542410285...	44 minut

19. What happens in the Target Group?

The stopped instance is marked unhealthy or removed (set to draining), and the remaining healthy instance continues handling traffic, a new target also appears.

20. Does your load test continue successfully?

Yes, the load test keeps running successfully because the load balancer routes traffic to the remaining and new instances.

IP address type	Load balancer
IPv4	CS6650L2-alb

3	2	0	0	0	1
Total targets	Healthy	Unhealthy	Unused	Initial	Draining
	0 Anomalous				

▶ **Distribution of targets by Availability Zone (AZ)**

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets | Monitoring | Health checks | Attributes | Tags

Registered targets (3) Info

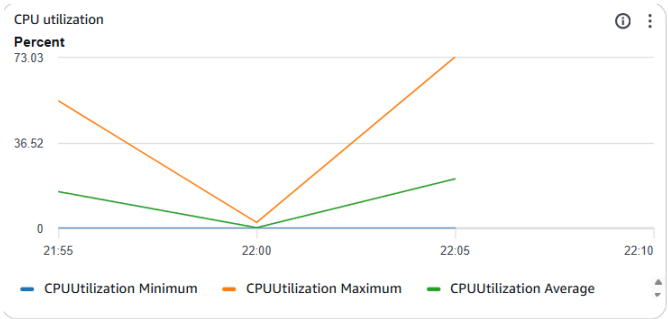
Anomaly mitigation: **Not applicable** | Deregister | Regis

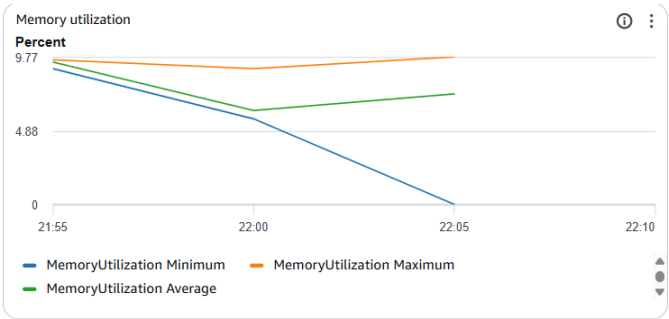
Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check set
Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

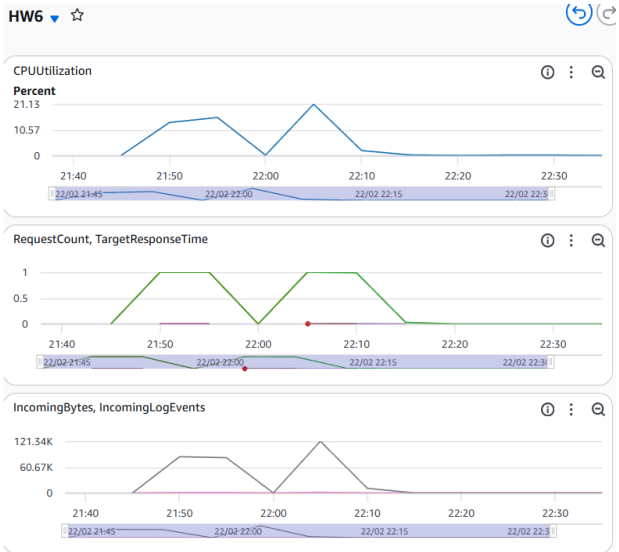
IP address	Port	Zone	Health status	Health status details	Administrative override	Overrid...	Anomaly de
172.31.8.240	8080	us-east-1b (...	Healthy	-	No override	No overrid...	Normal
172.31.95.72	8080	us-east-1c (...	Draining	Target deregistration is...	No override	No overrid...	Normal
172.31.24.40	8080	us-east-1d (...	Healthy	-	No override	No overrid...	Normal

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	151722	0	19	30	71	21.43	10	246	1604.82	900.8	0
	Aggregated	151722	0	19	30	71	21.43	10	246	1604.82	900.8	0





21. CloudWatch Monitoring



22. Load Testing Variations - Breaking Point: 500 users for 4 minutes

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	390145	0	44	110	190	54.59	10	516	1601.13	1831.3	0
	Aggregated	390145	0	44	110	190	54.59	10	516	1601.13	1831.3	0

23. Failure Scenario - Breaking Point: 100 users for 2 minutes - Stop All Instances

Task	Last status	Desired status	Task definition	Health status	Created at	Started by	Started
a4c02e18e6ed4eeca325b...	Deactivating	Stopped	CS6650L2-task:2	Unknown	1 hour ago	ecs-svc/25542410285...	1 hour
caf3b5aedfa1459e99c9a1...	Deactivating	Stopped	CS6650L2-task:2	Unknown	2 hours ago	ecs-svc/25542410285...	2 hours

Filter tasks by property or value | Any desired status | Any launch type | 1

Task	Last status	Desired status	Task definition	Health status	Created at	Started by	Started
354e572269724b0c887c...	Running	Running	CS6650L2-task:2	Unknown	38 seconds ago	ecs-svc/25542410285...	9 second
7ddfa4c56640484ba2724...	Running	Running	CS6650L2-task:2	Unknown	38 seconds ago	ecs-svc/25542410285...	6 second
a4c02e18e6ed4eeca325b...	Deactivating	Stopped	CS6650L2-task:2	Unknown	1 hour ago	ecs-svc/25542410285...	1 hour ag
caf3b5aedfa1459e99c9a1...	Deactivating	Stopped	CS6650L2-task:2	Unknown	2 hours ago	ecs-svc/25542410285...	2 hours a

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	121843	0	31	86	140	39.69	10	468	1602.22	1420.2	0
	Aggregated	121843	0	31	86	140	39.69	10	468	1602.22	1420.2	0

24. Explain how the system solved your Part II bottleneck

The system solved the bottleneck by adding more service instances automatically so the work was spread out instead of overloading one server.

25. Describe the role of each component (ALB, Target Group, Auto Scaling)

The ALB sends requests to the service, the Target Group keeps track of which instances are healthy, and Auto Scaling adds or removes instances based on CPU load.

26. Compare the trade-offs between this approach and vertical scaling

Horizontal scaling is more reliable and handles spikes better but is more complex, while vertical scaling is simpler but limited by how big one machine can get.

27. Predict scaling behavior for different load patterns

If load increases gradually the system will slowly add instances, but if there is a sudden spike it may lag briefly before scaling up enough to handle it.

28. terraform destroy -auto-approve